

Hartree-Fock program explanation

readinput.py

`readinput.py` processes the extended input file and extracts all data needed for the SCF and gradient calculations, including molecular geometry, basis-set definitions, integrals and integral derivatives. It converts the text blocks into structured arrays with 0-based indexing and applies the required symmetry and sign conventions.

```
In [ ]: class Atom:
        def __init__(self, label, Z, x, y, z):
            self.label = label
            self.Z = Z
            self.x = x
            self.y = y
            self.z = z

        class Molecule:
            def __init__(self, atoms, charge, nbasis, max_nc):
                self.atoms = atoms
                self.charge = charge
                self.nbasis = nbasis
                self.max_nc = max_nc
```

The following two classes, Atom and Molecule, provide the fundamental data structures used to store information from the input files.

The Atom class represents a single atomic center in the molecule:

- label: the chemical symbol of the atom (e.g., "H", "Li").
- Z: the nuclear charge, expressed as an integer.
- x, y, z: the Cartesian coordinates (in Ångström) of the atom in 3D space.

The Molecule class represents the complete molecular system described by the input file:

- atoms: a Python list of Atom objects, in the same order they appear in the input file.
- charge: the total molecular charge.
- nbasis: the total number of basis functions used for the SCF calculation.
- max_nc: the maximum number of primitive Gaussian functions contracted into any single basis function.

```
In [ ]: def _find_index(lines, pattern):
        pattern_lower = pattern.lower()

        for i, line in enumerate(lines):
            if line.lower().startswith(pattern_lower):
                return i

        raise ValueError("No line was found starting with: " + pattern)
```

The function `_find_index` is used to locate specific header lines inside the Hartree-Fock input files. The input files are structured in human-readable blocks (e.g., "number of atoms", "Overall charge").

`pattern_lower = pattern.lower()`

Converts the search pattern to lowercase. This allows the function to ignore differences in uppercase/lowercase when matching.

for i, line in enumerate(lines):

Iterates over each line in the file, keeping track of both the line content (line) and its position (i).

if line.lower().startswith(pattern_lower):

If the line begins with the pattern, the function returns the corresponding index.

```
In [ ]: def read_basic_input(path):  
        with open(path, "r") as f:  
            lines = []  
            for line in f:  
                stripped = line.strip()  
                if stripped:  
                    lines.append(stripped)
```

This block is responsible for reading the file and cleaning the raw text lines.

with open(path, "r") as f:

Opens the file located at path in read mode ("r").

lines = []

Initializes an empty list that will store all the non-empty lines.

for line in f:

Iterates over every line in the file object f.

stripped = line.strip()

Removes the whitespace characters (spaces, tabs, newline characters) that go before and after from each line.

if stripped:

This condition is True only if the line is not empty after stripping.

lines.append(stripped)

Adds the cleaned, non-empty line to the lines list.

Result: At the end of this block, lines is a list of strings where all empty lines have been removed, each line has no extra spaces at the beginning or end.

```
In [ ]: idx_na = _find_index(lines, "number of atoms")  
        natoms = int(lines[idx_na + 1])
```

This block extracts the number of atoms from the input.

idx_na = _find_index(lines, "number of atoms")

Calls the function _find_index to search for the line that starts with "number of atoms". The function returns the index of that line in the lines list.

natoms = int(lines[idx_na + 1])

The number of atoms is on the next line after the header. So we read line[idx_na + 1], and convert it to an integer using int(...).

```
In [ ]: idx_atoms_header = _find_index(lines, "Atom labels")  
        first_atom_line = idx_atoms_header + 1
```

Now we locate the start of the atomic coordinates block.

`idx_atoms_header = _find_index(lines, "Atom labels")` Finds the line that starts with "Atom labels". Again, the search is case-insensitive and returns the index of that header line.

`first_atom_line = idx_atoms_header + 1` The first atom is defined on the line immediately after this header, so we add 1 to move to the first data line.

```
In [ ]: atoms = []
        for j in range(natoms):
            parts = lines[first_atom_line + j].split()
            label = parts[0]
            Z = int(parts[1])
            x = float(parts[2])
            y = float(parts[3])
            z = float(parts[4])
            atom = Atom(label, Z, x, y, z)
            atoms.append(atom)
```

This loop reads each atom line and creates Atom objects.

```
atoms = []
```

Initializes an empty list that will store the Atom instances.

`for j in range(natoms):` Loops over each atom index from 0 to (*natoms* - 1). There should be exactly *natoms* consecutive lines describing atoms.

```
parts = lines[first_atom_line + j].split()
```

Takes the *j*-th atom line and splits it into separate components using whitespace as the separator.

For example, a line like:

"H 1 0.00000000 0.00000000 0.74080000" becomes:

["H", "1", "0.00000000", "0.00000000", "0.74080000"].

```
label = parts[0]
```

The first element is the chemical symbol.

```
Z = int(parts[1])
```

The second element is the nuclear charge, converted to an integer.

```
x = float(parts[2]), y = float(parts[3]), z = float(parts[4])
```

The remaining elements are the Cartesian coordinates of the atom, converted to floating-point numbers.

```
atom = Atom(label, Z, x, y, z)
```

Creates an *Atom* object using the previously defined *Atom* class.

```
atoms.append(atom)
```

Adds the newly created *Atom* to the atoms list in *Molecule*.

At the end of this loop, *atoms* is a list containing one *Atom* object for each atom defined in the input file.

```
In [ ]: idx_charge = _find_index(lines, "Overall charge")
        charge = int(lines[idx_charge + 1])
```

This block reads the total molecular charge.

```
idx_charge = _find_index(lines, "Overall charge")
```

Finds the line that starts with "Overall charge".

```
charge = int(lines[idx_charge + 1])
```

The value of the charge appears on the line immediately after the header. It is converted from string to

integer.

```
In [ ]: idx_nb = _find_index(lines, "Number of basis funcs")
        nbasis = int(lines[idx_nb + 1])
```

This block reads the total number of basis functions.

```
idx_nb = _find_index(lines, "Number of basis funcs")
```

Locates the header for the number of basis functions.

```
nbasis = int(lines[idx_nb + 1])
```

Reads the line immediately after the header, converts it to an integer, and stores it in nbasis.

```
In [ ]: idx_maxnc = _find_index(lines, "Maximum number of primitives")
        max_nc = int(lines[idx_maxnc + 1])
```

This block reads the maximum number of primitives in any contracted basis function.

```
idx_maxnc = _find_index(lines, "Maximum number of primitives")
```

Finds the header that marks this value.

```
max_nc = int(lines[idx_maxnc + 1])
```

Reads the next line, converts it to an integer, and stores it in max_nc.

```
In [ ]: mol = Molecule(atoms, charge, nbasis, max_nc)
        return mol
```

Finally, the function gathers all the data into a single *Molecule* object.

```
mol = Molecule(atoms, charge, nbasis, max_nc)
```

Creates an instance of the *Molecule* class using:

the list of Atom objects (atoms),

the total charge (charge),

the number of basis functions (nbasis),

and the maximum number of primitives (max_nc).

return mol Returns the constructed *Molecule*.

```
In [ ]: def read_integrals(path, nbasis):
```

This function extracts all the integrals from the extended input

It returns four NumPy arrays: the overlap matrix *S*, the kinetic matrix *T*, the nuclear attraction matrix *V*, and the two-electron tensor *eri*.

```
In [ ]: with open(path, "r") as f:
        lines = [line.strip() for line in f if line.strip()]
```

This block reads the input file and removes blank lines.

lines = [line.strip() for line in f if line.strip()] Builds a list of the stripped lines

```
In [ ]: idx_ov = _find_index(lines, "A. Overlap integrals")
        nS = int(lines[idx_ov + 1])
```

```
S = np.zeros((nbasis, nbasis), dtype=float)
```

This block locates the overlap section and allocates the S matrix.

idx_ov searches the line starting with "A. Overlap integrals", and nS is the number of overlap integrals. S is initialized as an nbasis-by-nbasis matrix of zeros.

```
In [ ]: for k in range(nS):
        parts = lines[idx_ov + 2 + k].split()
        mu = int(parts[0]) - 1
        nu = int(parts[1]) - 1
        val = float(parts[2])

        S[mu, nu] = val
        S[nu, mu] = val
```

This loop fills the overlap matrix.

Each line gives (mu, nu, value). The indices in the file are 1-based, so we subtract 1. $S[\text{mu}, \text{nu}]$ is equal to $S[\text{nu}, \text{mu}]$ since it is a hermitic matrix.

```
In [ ]: idx_kin = _find_index(lines, "B. Kinetic integrals")
        nT = int(lines[idx_kin + 1])

        T = np.zeros((nbasis, nbasis), dtype=float)

        for k in range(nT):
            parts = lines[idx_kin + 2 + k].split()
            mu = int(parts[0]) - 1
            nu = int(parts[1]) - 1
            val = float(parts[2])

            T[mu, nu] = val
            T[nu, mu] = val
```

This section reads the kinetic energy integrals.

It follows the same pattern as the overlap block: locate the header, read the count, allocate T , and fill it symmetrically.

```
In [ ]: idx_v = _find_index(lines, "C. Nuclear Attraction integrals")
        nV = int(lines[idx_v + 1])

        V = np.zeros((nbasis, nbasis), dtype=float)

        for k in range(nV):
            parts = lines[idx_v + 2 + k].split()
            mu = int(parts[0]) - 1
            nu = int(parts[1]) - 1
            val = float(parts[2])

            V[mu, nu] = val
            V[nu, mu] = val
```

This section reads the electron-nuclear attraction integrals.

Like the previous blocks, it builds a symmetric nbasis-by-nbasis matrix V from the input lines.

```
In [ ]: idx_eri = _find_index(lines, "D. Two-Electron integrals")
        nERI = int(lines[idx_eri + 1])
```

```
eri = np.zeros((nbasis, nbasis, nbasis, nbasis), dtype=float)

for k in range(nERI):
    parts = lines[idx_eri + 2 + k].split()
    mu = int(parts[0]) - 1
    nu = int(parts[1]) - 1
    lam = int(parts[2]) - 1
    sig = int(parts[3]) - 1
    val = float(parts[4])
```

```
eri[mu, nu, lam, sig] = val
eri[mu, nu, sig, lam] = val
eri[nu, mu, lam, sig] = val
eri[nu, mu, sig, lam] = val
eri[lam, sig, mu, nu] = val
eri[lam, sig, nu, mu] = val
eri[sig, lam, mu, nu] = val
eri[sig, lam, nu, mu] = val
```

This block reads the two-electron integrals.

Each entry specifies (mu, nu, lam, sig, value). The tensor is four-dimensional, and all symmetry-related permutations are filled so that $(\mu\nu|\lambda\sigma)$ equals its equivalent reorderings.

```
In [ ]: return S, T, V, eri
```

The function returns the four arrays that will be used later in the SCF procedure.

```
In [ ]: def read_basis_block(path, nbasis):
```

This function reads the basis-set block and returns both the basis-function data and the basis-to-atom mapping.

It returns a tuple (basis, mu_to_atom), where basis stores primitives per function and mu_to_atom maps each basis function to its atom index.

```
In [ ]: with open(path, "r") as f:
        lines = [line.strip() for line in f if line.strip()]
```

This block reads the file and builds a list of non-empty, stripped lines.

```
In [ ]: idx = _find_index(lines, "Basis set:")
        idx += 1

        basis = [None] * nbasis
        mu_to_atom = [None] * nbasis
```

This locates the basis-set header and initializes the output lists.

```
In [ ]: for k in range(nbasis):
        parts = lines[idx].split()
        mu = int(parts[0]) - 1      # 0-based
        atom_index = int(parts[2]) - 1 # 0-based
        mu_to_atom[mu] = atom_index

        idx += 1
        nprim = int(lines[idx])
        idx += 1

        primitives = []
        for i in range(nprim):
```

```

        zeta_str, coeff_str = lines[idx].split()
        primitives.append((float(zeta_str), float(coeff_str)))
        idx += 1

    basis[mu] = {
        "atom_index": atom_index,
        "primitives": primitives,
    }

```

This loop parses one basis-function block at a time. It reads the function index (μ) and atom index (`atom_index`), then reads the number of primitives and collects each `(zeta, coeff)` pair before storing them in the `basis` list.

```

In [ ]: if any(x is None for x in mu_to_atom):
        raise ValueError("Failed to build mu_to_atom from Basis set block.")

```

This is a safety check: any remaining None means a basis function was not assigned.

```

In [ ]: return basis, mu_to_atom

```

This function returns two objects: `mu_to_atom` and `basis`.

The array `mu_to_atom` maps each basis function index μ (0-based) to the atom on which it is centered.

For example, if the input contains:

```

1 H 1
2 Li 2
3 Li 2

```

the function returns:

```
mu_to_atom = [0, 1, 1]
```

meaning that basis function 0 is centered on atom 0 (H), and basis functions 1 and 2 are centered on atom 1 (Li).

The list `basis` stores the full definition of each basis function.

Each entry `basis[ $\mu$ ]` contains:

- the index of the atom on which the basis function is centered (`atom_index`)
- the list of primitive Gaussians defining the contraction (`primitives`)

For the same input, the function returns:

```

basis = [ { atom_index: 0, primitives: [ (3.42525091, 0.27693436), (0.62391373, 0.26783885), (0.16885540, 0.08347367) ] },
          { atom_index: 1, primitives: [ (16.11957500, 0.88485629), (2.93620070, 0.85579446), (0.79465050, 0.26671375) ] },
          { atom_index: 1, primitives: [ (0.63628970, -0.05075852), (0.14786010, 0.06789364), (0.04808870, 0.05124034) ] } ]

```

This means that basis function 0 is a contracted Gaussian centered on atom 0 (H), while basis functions 1 and 2 are two different contracted Gaussian functions centered on atom 1 (Li), each defined as a linear combination of three primitive Gaussians.

```

In [ ]: def read_derivatives(path, nbasis, natoms, mu_to_atom):

```

This function reads all derivative blocks from an extended input file.

It returns derivative tensors for overlap (dS), kinetic (dT), nuclear attraction (dV), and two-electron integrals ($dERI$), indexed by atom and Cartesian component.

```
In [ ]: with open(path, "r") as f:
        lines = [line.strip() for line in f if line.strip()]
```

The file is read once and blank lines are removed.

```
In [ ]: idx = _find_index(lines, "E. Derivatives of overlap integrals")
nvec = int(lines[idx + 1])
start = idx + 2
while start < len(lines) and not lines[start][0].isdigit():
    start += 1

dS = np.zeros((natoms, nbasis, nbasis, 3), dtype=float) # dS[A, mu, nu, xyz]

for k in range(nvec):
    parts = lines[start + k].split()
    mu = int(parts[0]) - 1
    nu = int(parts[1]) - 1
    dx = float(parts[2])
    dy = float(parts[3])
    dz = float(parts[4])
    vec = (dx, dy, dz)

    atom_mu = mu_to_atom[mu]
    atom_nu = mu_to_atom[nu]

    dS[atom_mu, mu, nu, :] = vec
    dS[atom_mu, nu, mu, :] = vec

    if atom_nu != atom_mu:
        dS[atom_nu, mu, nu, :] = (-dx, -dy, -dz)
        dS[atom_nu, nu, mu, :] = (-dx, -dy, -dz)
```

This code block reads the section “*E. Derivatives of overlap integrals*” from the extended input and stores the nuclear derivatives of the overlap matrix in a tensor

$$dS[A, \mu, \nu, xyz] = \frac{\partial S_{\mu\nu}}{\partial \mathbf{R}_A}.$$

First, the header of the block is located and the number of derivative vectors (`nvec`) is read. The parser then skips non-numeric lines until it reaches the first data line. A zero-initialized array of shape `(natoms, nbasis, nbasis, 3)` is created to hold the x, y, and z components of the derivatives for each atom and pair of basis functions.

Each derivative line provides the indices of two basis functions (μ, ν) and a vector `(dx, dy, dz)`. According to the input specification, this vector is the derivative with respect to the coordinates of the atom on which basis function μ is centered. Using the `mu_to_atom` mapping, the vector is assigned to the corresponding atom and stored symmetrically for `( $\mu, \nu$ )` and `( $\nu, \mu$ )` since the overlap matrix is symmetric. If μ and ν are centered on different atoms, the same derivative with opposite sign is assigned to the atom on which ν is centered, enforcing translational invariance of the overlap integrals.

```
In [ ]: idx = _find_index(lines, "F. Derivatives of kinetic energy integrals")
nvec = int(lines[idx + 1])
start = idx + 2
while start < len(lines) and not lines[start][0].isdigit():
    start += 1

dT = np.zeros((natoms, nbasis, nbasis, 3), dtype=float) # dT[A, mu, nu, xyz]
```



```

for k in range(nvec):
    parts = lines[start + k].split()
    mu = int(parts[0]) - 1
    nu = int(parts[1]) - 1
    dx = float(parts[2])
    dy = float(parts[3])
    dz = float(parts[4])
    vec = (dx, dy, dz)

    atom_mu = mu_to_atom[mu]
    atom_nu = mu_to_atom[nu]

    dT[atom_mu, mu, nu, :] = vec
    dT[atom_mu, nu, mu, :] = vec

    if atom_nu != atom_mu:
        dT[atom_nu, mu, nu, :] = (-dx, -dy, -dz)
        dT[atom_nu, nu, mu, :] = (-dx, -dy, -dz)

```

This block works in the same way as for the overlap-derivative handling, but for kinetic energy derivatives.

```

In [ ]: idx = _find_index(lines, "G. Derivatives of Nucleus-electron energy integrals")
nvec = int(lines[idx + 1])
start = idx + 2
while start < len(lines) and not lines[start][0].isdigit():
    start += 1

dV = np.zeros((natoms, nbasis, nbasis, 3), dtype=float) # dV[A, mu, nu, xyz]

for k in range(nvec):
    parts = lines[start + k].split()
    mu = int(parts[0]) - 1
    nu = int(parts[1]) - 1
    atom = int(parts[2]) - 1
    dx = float(parts[3])
    dy = float(parts[4])
    dz = float(parts[5])

    dV[atom, mu, nu, 0] = dx
    dV[atom, mu, nu, 1] = dy
    dV[atom, mu, nu, 2] = dz

    dV[atom, nu, mu, 0] = dx
    dV[atom, nu, mu, 1] = dy
    dV[atom, nu, mu, 2] = dz

```

This section reads derivatives of the electron-nuclear attraction integrals.

The atom index is provided explicitly in the file, so the derivatives are stored directly in *dV* for that atom, with symmetry enforced for (mu, nu).

```

In [ ]: idx = _find_index(lines, "H. Derivatives of two-electron integrals")
nvec = int(lines[idx + 1])
start = idx + 2
while start < len(lines) and not lines[start][0].isdigit():
    start += 1

dERI = np.zeros((natoms, nbasis, nbasis, nbasis, nbasis, 3), dtype=float) # dERI[A, mu, nu,

for k in range(nvec):
    parts = lines[start + k].split()
    mu = int(parts[0]) - 1

```

```

nu = int(parts[1]) - 1
lam = int(parts[2]) - 1
sig = int(parts[3]) - 1
atom = int(parts[4]) - 1
dx = float(parts[5])
dy = float(parts[6])
dz = float(parts[7])

```

```

for (a, b, c, d) in [
    (mu, nu, lam, sig),
    (mu, nu, sig, lam),
    (nu, mu, lam, sig),
    (nu, mu, sig, lam),
    (lam, sig, mu, nu),
    (lam, sig, nu, mu),
    (sig, lam, mu, nu),
    (sig, lam, nu, mu),
]:
    dERI[atom, a, b, c, d, 0] = dx
    dERI[atom, a, b, c, d, 1] = dy
    dERI[atom, a, b, c, d, 2] = dz

```

This block handles two-electron integral derivatives.

Each line specifies (mu, nu, lam, sig, atom, dx, dy, dz). All symmetry-related index permutations are filled so later code can access dERI consistently.

```
In [ ]: return dS, dT, dV, dERI
```

The function returns the four derivative tensors for use in gradient evaluation.

integrals.py

This program implements analytic calculations of s-type Gaussian integrals and their derivatives. All coordinates are converted to Bohr before evaluation. The formulas below are written for primitive Gaussians and then contracted with the basis coefficients.

```
In [ ]: import math
import numpy as np

ANGSTROM_TO_BOHR = 1.8897261246257702
```

NumPy is used for vector arithmetic, and the coordinate conversion constant is

$$R_{\text{bohr}} = R_{\text{angstrom}} \times 1.8897261246257702.$$

```
In [ ]: def _overlap_ss(alpha, beta, RA, RB):
    p = alpha + beta
    mu = alpha * beta / p
    rab2 = np.dot(RA - RB, RA - RB)
    return (np.pi / p) ** 1.5 * np.exp(-mu * rab2)
```

This is the analytic overlap integral between two s-type primitive Gaussians. (44)

$$p = \alpha + \beta$$

$$\mu = \frac{\alpha\beta}{p}$$

$$r_{AB}^2 = |\mathbf{R}_A - \mathbf{R}_B|^2$$

$$S_{k,\mu,l,\nu} = \left(\frac{\pi}{p}\right)^{3/2} \exp(-\mu r_{AB}^2)$$

```
In [ ]: def _kinetic_ss(alpha, beta, RA, RB):
    p = alpha + beta
    mu = alpha * beta / p
    rab2 = np.dot(RA - RB, RA - RB)
    overlap = _overlap_ss(alpha, beta, RA, RB)
    return mu * (3.0 - 2.0 * mu * rab2) * overlap
```

This is the s-type kinetic energy integral. (46)

$$T_{k,\mu,l,\nu} = \mu(3 - 2\mu r_{AB}^2) S_{k,\mu,l,\nu}$$

```
In [ ]: def _boys0(t):
    if t < 1e-8:
        return 1.0
    return 0.5 * math.sqrt(math.pi / t) * math.erf(math.sqrt(t))

def _boys1(t):
    if t < 1e-8:
        return 1.0 / 3.0
    return (_boys0(t) - math.exp(-t)) / (2.0 * t)
```

These are the Boys functions used in nuclear attraction and two-electron integrals. (52)

$$F_0(t) = \frac{1}{2} \sqrt{\pi/t} \operatorname{erf}(\sqrt{t})$$

$$F_1(t) = \frac{F_0(t) - e^{-t}}{2t}$$

For small t , the series expansions are used for numerical stability:

$$F_0(t) \approx 1 \text{ and } F_1(t) \approx 1/3$$

```
In [ ]: def _nuclear_attraction_ss(alpha, beta, RA, RB, RC, ZC):
    p = alpha + beta
    RP = (alpha * RA + beta * RB) / p
    rpc2 = np.dot(RP - RC, RP - RC)
    overlap = _overlap_ss(alpha, beta, RA, RB)
    return 2.0 * -ZC * math.sqrt(p / np.pi) * overlap * _boys0(p * rpc2)

def _nuclear_attraction_ss1(alpha, beta, RA, RB, RC, ZC):
    p = alpha + beta
    RP = (alpha * RA + beta * RB) / p
    rpc2 = np.dot(RP - RC, RP - RC)
    overlap = _overlap_ss(alpha, beta, RA, RB)
    return 2.0 * -ZC * math.sqrt(p / np.pi) * overlap * _boys1(p * rpc2)
```

These are the s-type electron-nuclear attraction integrals. The Gaussian product center is (43)

$$\mathbf{R}_P = (\alpha \mathbf{R}_A + \beta \mathbf{R}_B) / p$$

The integral is

$$V_{k,\mu,l,\nu,C} = -2Z_C \sqrt{\frac{p}{\pi}} S_{k,\mu,l,\nu} F_0(pr_{PC}^2)$$

and $V_{k,\mu,l,\nu,C}^{(1)}$ is the same formula using F_1 .

```
In [ ]: def _eri_ss(alpha, beta, gamma, delta, RA, RB, RC, RD):
    p = alpha + beta
    q = gamma + delta
    mu = alpha * beta / p
    nu = gamma * delta / q
    rab2 = np.dot(RA - RB, RA - RB)
    rcd2 = np.dot(RC - RD, RC - RD)
    RP = (alpha * RA + beta * RB) / p
    RQ = (gamma * RC + delta * RD) / q
    rpq2 = np.dot(RP - RQ, RP - RQ)
    K_ab = math.sqrt(2.0) * (math.pi ** (5.0 / 4.0) / p) * math.exp(-mu * rab2)
    K_cd = math.sqrt(2.0) * (math.pi ** (5.0 / 4.0) / q) * math.exp(-nu * rcd2)
    return (K_ab * K_cd / math.sqrt(p + q)) * _boys0((p * q / (p + q)) * rpq2)

def _eri_ss1(alpha, beta, gamma, delta, RA, RB, RC, RD):
    p = alpha + beta
    q = gamma + delta
    mu = alpha * beta / p
    nu = gamma * delta / q
    rab2 = np.dot(RA - RB, RA - RB)
    rcd2 = np.dot(RC - RD, RC - RD)
    RP = (alpha * RA + beta * RB) / p
    RQ = (gamma * RC + delta * RD) / q
    rpq2 = np.dot(RP - RQ, RP - RQ)
    K_ab = math.sqrt(2.0) * (math.pi ** (5.0 / 4.0) / p) * math.exp(-mu * rab2)
    K_cd = math.sqrt(2.0) * (math.pi ** (5.0 / 4.0) / q) * math.exp(-nu * rcd2)
    return (K_ab * K_cd / math.sqrt(p + q)) * _boys1((p * q / (p + q)) * rpq2)
```

These are the two-electron repulsion integrals (s,s|s,s).

$$p = \alpha + \beta, \quad q = \gamma + \delta$$

$$\mu = \alpha\beta/p, \quad \nu = \gamma\delta/q$$

$$r_{AB}^2 = |\mathbf{R}_A - \mathbf{R}_B|^2, \quad r_{CD}^2 = |\mathbf{R}_C - \mathbf{R}_D|^2$$

$$\mathbf{R}_P = (\alpha\mathbf{R}_A + \beta\mathbf{R}_B)/p, \quad \mathbf{R}_Q = (\gamma\mathbf{R}_C + \delta\mathbf{R}_D)/q$$

$$r_{PQ}^2 = |\mathbf{R}_P - \mathbf{R}_Q|^2$$

The prefactors are

$$K_{ab} = \sqrt{2} \frac{\pi^{5/4}}{p} \exp(-\mu r_{AB}^2), \quad K_{cd} = \sqrt{2} \frac{\pi^{5/4}}{q} \exp(-\nu r_{CD}^2)$$

The integrals are

$$(ab|cd) = \frac{K_{ab}K_{cd}}{\sqrt{p+q}} F_0 \left(\frac{pq}{p+q} r_{PQ}^2 \right)$$

and $(ab|cd)^{(1)}$ uses F_1 .

```
In [ ]: def compute_overlap(mol, basis):
    nbasis = len(basis)
    S = np.zeros((nbasis, nbasis), dtype=float)

    coords = (
        np.array([[atom.x, atom.y, atom.z] for atom in mol.atoms], dtype=float)
        * ANGSTROM_TO_BOHR
    )
    for mu in range(nbasis):
        info_mu = basis[mu]
```

```

RA = coords[info_mu["atom_index"]]
prims_mu = info_mu["primitives"]
for nu in range(mu + 1):
    info_nu = basis[nu]
    RB = coords[info_nu["atom_index"]]
    prims_nu = info_nu["primitives"]

    total = 0.0
    for alpha, coeff_a in prims_mu:
        for beta, coeff_b in prims_nu:
            total += coeff_a * coeff_b * _overlap_ss(alpha, beta, RA, RB)
    S[mu, nu] = total
    S[nu, mu] = total

return S

```

This builds the contracted overlap matrix

$$S_{\mu\nu} = \sum_{a \in \mu} \sum_{b \in \nu} c_a c_b S_{ss}(\alpha_a, \beta_b).$$

The matrix is symmetric, so only the lower triangle is computed and mirrored.

```

In [ ]: def compute_kinetic(mol, basis):
    nbasis = len(basis)
    T = np.zeros((nbasis, nbasis), dtype=float)

    coords = (
        np.array([[atom.x, atom.y, atom.z] for atom in mol.atoms], dtype=float)
        * ANGSTROM_TO_BOHR
    )
    for mu in range(nbasis):
        info_mu = basis[mu]
        RA = coords[info_mu["atom_index"]]
        prims_mu = info_mu["primitives"]
        for nu in range(mu + 1):
            info_nu = basis[nu]
            RB = coords[info_nu["atom_index"]]
            prims_nu = info_nu["primitives"]

            total = 0.0
            for alpha, coeff_a in prims_mu:
                for beta, coeff_b in prims_nu:
                    total += coeff_a * coeff_b * _kinetic_ss(alpha, beta, RA, RB)
            T[mu, nu] = total
            T[nu, mu] = total

    return T

```

This builds the contracted kinetic energy matrix

$$T_{\mu\nu} = \sum_{a \in \mu} \sum_{b \in \nu} c_a c_b T_{ss}(\alpha_a, \beta_b),$$

again using symmetry to fill the upper triangle.

```

In [ ]: def compute_nuclear_attraction(mol, basis):
    nbasis = len(basis)
    V = np.zeros((nbasis, nbasis), dtype=float)

    coords = (
        np.array([[atom.x, atom.y, atom.z] for atom in mol.atoms], dtype=float)
        * ANGSTROM_TO_BOHR
    )
    charges = np.array([atom.Z for atom in mol.atoms], dtype=float)

```

```

for mu in range(nbasis):
    info_mu = basis[mu]
    RA = coords[info_mu["atom_index"]]
    prims_mu = info_mu["primitives"]
    for nu in range(mu + 1):
        info_nu = basis[nu]
        RB = coords[info_nu["atom_index"]]
        prims_nu = info_nu["primitives"]

        total = 0.0
        for alpha, coeff_a in prims_mu:
            for beta, coeff_b in prims_nu:
                for C, ZC in enumerate(charges):
                    RC = coords[C]
                    total += coeff_a * coeff_b * _nuclear_attraction_ss(
                        alpha, beta, RA, RB, RC, ZC
                    )
        V[mu, nu] = total
        V[nu, mu] = total

return V

```

This builds the nuclear attraction matrix

$$V_{\mu\nu} = \sum_{a \in \mu} \sum_{b \in \nu} \sum_C c_a c_b V_{ss}(\alpha_a, \beta_b; C),$$

summing over all nuclei with charges Z_C .

```

In [ ]: def compute_two_electron(mol, basis):
    nbasis = len(basis)
    eri = np.zeros((nbasis, nbasis, nbasis, nbasis), dtype=float)

    coords = (
        np.array([[atom.x, atom.y, atom.z] for atom in mol.atoms], dtype=float)
        * ANGSTROM_TO_BOHR
    )
    for mu in range(nbasis):
        info_mu = basis[mu]
        RA = coords[info_mu["atom_index"]]
        prims_mu = info_mu["primitives"]
        for nu in range(mu + 1):
            info_nu = basis[nu]
            RB = coords[info_nu["atom_index"]]
            prims_nu = info_nu["primitives"]
            for lam in range(nbasis):
                info_lam = basis[lam]
                RC = coords[info_lam["atom_index"]]
                prims_lam = info_lam["primitives"]
                for sig in range(lam + 1):
                    info_sig = basis[sig]
                    RD = coords[info_sig["atom_index"]]
                    prims_sig = info_sig["primitives"]

                    total = 0.0
                    for alpha, coeff_a in prims_mu:
                        for beta, coeff_b in prims_nu:
                            for gamma, coeff_c in prims_lam:
                                for delta, coeff_d in prims_sig:
                                    total += (
                                        coeff_a
                                        * coeff_b
                                        * coeff_c
                                        * coeff_d
                                        * _eri_ss(
                                            alpha, beta, gamma, delta, RA, RB, RC, RD

```

```

        eri[mu, nu, lam, sig] = total
        eri[nu, mu, lam, sig] = total
        eri[mu, nu, sig, lam] = total
        eri[nu, mu, sig, lam] = total
        eri[lam, sig, mu, nu] = total
        eri[sig, lam, mu, nu] = total
        eri[lam, sig, nu, mu] = total
        eri[sig, lam, nu, mu] = total

    return eri

```

This builds the 4-index electron repulsion tensor

$$(\mu\nu|\lambda\sigma) = \sum_{a\in\mu} \sum_{b\in\nu} \sum_{c\in\lambda} \sum_{d\in\sigma} c_a c_b c_c c_d (ab|cd).$$

All eight permutational symmetries are filled to keep later contractions simple.

```

In [ ]: def compute_overlap_derivatives(mol, basis):
    natoms = len(mol.atoms)
    nbasis = len(basis)
    dS = np.zeros((natoms, nbasis, nbasis, 3), dtype=float)

    coords = (
        np.array([[atom.x, atom.y, atom.z] for atom in mol.atoms], dtype=float)
        * ANGSTROM_TO_BOHR
    )
    for mu in range(nbasis):
        info_mu = basis[mu]
        A = info_mu["atom_index"]
        RA = coords[A]
        prims_mu = info_mu["primitives"]
        for nu in range(mu + 1):
            info_nu = basis[nu]
            B = info_nu["atom_index"]
            RB = coords[B]
            prims_nu = info_nu["primitives"]

            dA = np.zeros(3, dtype=float)
            dB = np.zeros(3, dtype=float)
            for alpha, coeff_a in prims_mu:
                for beta, coeff_b in prims_nu:
                    p = alpha + beta
                    RP = (alpha * RA + beta * RB) / p
                    S = _overlap_ss(alpha, beta, RA, RB)
                    pref = coeff_a * coeff_b
                    dA += pref * (2.0 * alpha * (RP - RA) * S)
                    dB += pref * (2.0 * beta * (RP - RB) * S)

            dS[A, mu, nu] += dA
            dS[B, mu, nu] += dB
            dS[A, nu, mu] = dS[A, mu, nu]
            dS[B, nu, mu] = dS[B, mu, nu]

    return dS

```

This computes analytic overlap derivatives with respect to nuclear coordinates.

For a primitive pair, the derivative is

$$\frac{\partial S_{ss}}{\partial \mathbf{R}_A} = 2\alpha(\mathbf{R}_P - \mathbf{R}_A)S_{ss},$$

$$\frac{\partial S_{ss}}{\partial \mathbf{R}_B} = 2\beta(\mathbf{R}_P - \mathbf{R}_B)S_{ss}.$$

The code accumulates both atom contributions and enforces symmetry.

```
In [ ]: def compute_kinetic_derivatives(mol, basis):
    natoms = len(mol.atoms)
    nbasis = len(basis)
    dT = np.zeros((natoms, nbasis, nbasis, 3), dtype=float)

    coords = (
        np.array([[atom.x, atom.y, atom.z] for atom in mol.atoms], dtype=float)
        * ANGSTROM_TO_BOHR
    )
    for mu in range(nbasis):
        info_mu = basis[mu]
        A = info_mu["atom_index"]
        RA = coords[A]
        prims_mu = info_mu["primitives"]
        for nu in range(mu + 1):
            info_nu = basis[nu]
            B = info_nu["atom_index"]
            RB = coords[B]
            prims_nu = info_nu["primitives"]

            dA = np.zeros(3, dtype=float)
            dB = np.zeros(3, dtype=float)
            for alpha, coeff_a in prims_mu:
                for beta, coeff_b in prims_nu:
                    p = alpha + beta
                    RP = (alpha * RA + beta * RB) / p
                    mu_exp = alpha * beta / p
                    S = _overlap_ss(alpha, beta, RA, RB)
                    T = _kinetic_ss(alpha, beta, RA, RB)
                    pref = coeff_a * coeff_b
                    dA += pref * (2.0 * alpha * (RP - RA) * (T + 2.0 * mu_exp * S))
                    dB += pref * (2.0 * beta * (RP - RB) * (T + 2.0 * mu_exp * S))

            dT[A, mu, nu] += dA
            dT[B, mu, nu] += dB
            dT[A, nu, mu] = dT[A, mu, nu]
            dT[B, nu, mu] = dT[B, mu, nu]

    return dT
```

The kinetic derivative uses the formula

$$\frac{\partial T_{ss}}{\partial \mathbf{R}_A} = 2\alpha(\mathbf{R}_P - \mathbf{R}_A)(T_{ss} + 2\mu S_{ss}),$$

$$\frac{\partial T_{ss}}{\partial \mathbf{R}_B} = 2\beta(\mathbf{R}_P - \mathbf{R}_B)(T_{ss} + 2\mu S_{ss}).$$

```
In [ ]: def compute_nuclear_attraction_derivatives(mol, basis):
    natoms = len(mol.atoms)
    nbasis = len(basis)
    dV = np.zeros((natoms, nbasis, nbasis, 3), dtype=float)

    coords = (
        np.array([[atom.x, atom.y, atom.z] for atom in mol.atoms], dtype=float)
        * ANGSTROM_TO_BOHR
    )
    charges = np.array([atom.Z for atom in mol.atoms], dtype=float)

    for mu in range(nbasis):
        info_mu = basis[mu]
```



```

A = info_mu["atom_index"]
RA = coords[A]
prims_mu = info_mu["primitives"]
for nu in range(mu + 1):
    info_nu = basis[nu]
    B = info_nu["atom_index"]
    RB = coords[B]
    prims_nu = info_nu["primitives"]

    dA = np.zeros(3, dtype=float)
    dB = np.zeros(3, dtype=float)
    dC = np.zeros((natoms, 3), dtype=float)

    for alpha, coeff_a in prims_mu:
        for beta, coeff_b in prims_nu:
            p = alpha + beta
            RP = (alpha * RA + beta * RB) / p
            pref = coeff_a * coeff_b
            for C, ZC in enumerate(charges):
                RC = coords[C]
                V = _nuclear_attraction_ss(alpha, beta, RA, RB, RC, ZC)
                V1 = _nuclear_attraction_ss1(alpha, beta, RA, RB, RC, ZC)
                dA += pref * (2.0 * alpha * ((RP - RA) * V - (RP - RC) * V1))
                dB += pref * (2.0 * beta * ((RP - RB) * V - (RP - RC) * V1))
                dC[C] += pref * (2.0 * p * (RP - RC) * V1)

    dV[A, mu, nu] += dA
    dV[B, mu, nu] += dB
    for C in range(natoms):
        dV[C, mu, nu] += dC[C]
    dV[:, nu, mu] = dV[:, mu, nu]

return dV

```

The nuclear attraction derivatives use

$$\frac{\partial V_{ss}}{\partial \mathbf{R}_A} = 2\alpha \left[(\mathbf{R}_P - \mathbf{R}_A) V_{ss} - (\mathbf{R}_P - \mathbf{R}_C) V_{ss}^{(1)} \right],$$

$$\frac{\partial V_{ss}}{\partial \mathbf{R}_B} = 2\beta \left[(\mathbf{R}_P - \mathbf{R}_B) V_{ss} - (\mathbf{R}_P - \mathbf{R}_C) V_{ss}^{(1)} \right],$$

$$\frac{\partial V_{ss}}{\partial \mathbf{R}_C} = 2p(\mathbf{R}_P - \mathbf{R}_C) V_{ss}^{(1)}.$$

Each nucleus C contributes, so the code accumulates a derivative vector for every atom.

```

In [ ]: def compute_two_electron_derivatives(mol, basis):
    natoms = len(mol.atoms)
    nbasis = len(basis)
    dERI = np.zeros((natoms, nbasis, nbasis, nbasis, nbasis, 3), dtype=float)

    coords = (
        np.array([[atom.x, atom.y, atom.z] for atom in mol.atoms], dtype=float)
        * ANGSTROM_TO_BOHR
    )

    for mu in range(nbasis):
        info_mu = basis[mu]
        A = info_mu["atom_index"]
        RA = coords[A]
        prims_mu = info_mu["primitives"]
        for nu in range(mu + 1):
            info_nu = basis[nu]
            B = info_nu["atom_index"]
            RB = coords[B]

```

```

prims_nu = info_nu["primitives"]
for lam in range(nbasis):
    info_lam = basis[lam]
    C = info_lam["atom_index"]
    RC = coords[C]
    prims_lam = info_lam["primitives"]
    for sig in range(lam + 1):
        info_sig = basis[sig]
        D = info_sig["atom_index"]
        RD = coords[D]
        prims_sig = info_sig["primitives"]

    dA = np.zeros(3, dtype=float)
    dB = np.zeros(3, dtype=float)
    dC = np.zeros(3, dtype=float)
    dD = np.zeros(3, dtype=float)

    for alpha, coeff_a in prims_mu:
        for beta, coeff_b in prims_nu:
            p = alpha + beta
            RP = (alpha * RA + beta * RB) / p
            for gamma, coeff_c in prims_lam:
                for delta, coeff_d in prims_sig:
                    q = gamma + delta
                    RQ = (gamma * RC + delta * RD) / q
                    W = (p * RP + q * RQ) / (p + q)
                    eri = _eri_ss(
                        alpha, beta, gamma, delta, RA, RB, RC, RD
                    )
                    eri1 = _eri_ss1(
                        alpha, beta, gamma, delta, RA, RB, RC, RD
                    )
                    pref = coeff_a * coeff_b * coeff_c * coeff_d
                    dA += pref * (
                        2.0 * alpha * ((RP - RA) * eri + (W - RP) * eri1)
                    )
                    dB += pref * (
                        2.0 * beta * ((RP - RB) * eri + (W - RP) * eri1)
                    )
                    dC += pref * (
                        2.0 * gamma * ((RQ - RC) * eri + (W - RQ) * eri1)
                    )
                    dD += pref * (
                        2.0 * delta * ((RQ - RD) * eri + (W - RQ) * eri1)
                    )

    acc = {}
    for idx, vec in ((A, dA), (B, dB), (C, dC), (D, dD)):
        if idx in acc:
            acc[idx] += vec
        else:
            acc[idx] = vec.copy()

    for Aidx, vec in acc.items():
        dERI[Aidx, mu, nu, lam, sig] = vec
        dERI[Aidx, nu, mu, lam, sig] = vec
        dERI[Aidx, mu, nu, sig, lam] = vec
        dERI[Aidx, nu, mu, sig, lam] = vec
        dERI[Aidx, lam, sig, mu, nu] = vec
        dERI[Aidx, sig, lam, mu, nu] = vec
        dERI[Aidx, lam, sig, nu, mu] = vec
        dERI[Aidx, sig, lam, nu, mu] = vec

return dERI

```

The two-electron derivative uses the center

$$\mathbf{W} = \frac{p\mathbf{R}_P + q\mathbf{R}_Q}{p+q}$$

and the formula

$$\frac{\partial(ab|cd)}{\partial\mathbf{R}_A} = 2\alpha [(\mathbf{R}_P - \mathbf{R}_A)(ab|cd) + (\mathbf{W} - \mathbf{R}_P)(ab|cd)^{(1)}],$$

with analogous expressions for $\mathbf{R}_B, \mathbf{R}_C, \mathbf{R}_D$ using β, γ, δ and centers $\mathbf{R}_P, \mathbf{R}_Q$.

After contraction, all symmetry-related tensor entries are filled. When multiple basis functions are centered on the same atom, their derivative contributions are accumulated.

scf.py

`scf.py` implements the RHF self-consistent field procedure and the matrix builders used by the SCF loop. The key equations used throughout are:

- Core Hamiltonian: $H_{\mu\nu} = T_{\mu\nu} + V_{\mu\nu}$
- Fock matrix: $F_{\mu\nu} = H_{\mu\nu} + J_{\mu\nu} - \frac{1}{2}K_{\mu\nu}$
- Electronic energy: $E_{\text{elec}} = \frac{1}{2} \sum_{\mu\nu} P_{\mu\nu} (H_{\mu\nu} + F_{\mu\nu})$

```
In [ ]: import numpy as np
```

```
ANGSTROM_TO_BOHR = 1.8897261246257702
```

NumPy provides linear algebra utilities. The coordinate conversion used later is:

$$R_{\text{bohr}} = R_{\text{angstrom}} \times 1.8897261246257702.$$

```
In [ ]: def build_Hcore(T, V):  
        return T + V
```

This constructs the core Hamiltonian directly from the kinetic and nuclear attraction matrices:

$$H_{\mu\nu} = T_{\mu\nu} + V_{\mu\nu}.$$

```
In [ ]: def build_S_inv_sqrt(S):  
        evals, evecs = np.linalg.eigh(S)  
        inv_sqrt_evals = np.diag(1.0 / np.sqrt(evals))  
        S_inv_sqrt = evecs @ inv_sqrt_evals @ evecs.T  
        return S_inv_sqrt
```

The overlap matrix is diagonalized and inverted by square root:

$$S = U(s_i)U^T$$

$$S^{-1/2} = U(s_i^{-1/2})U^T.$$

```
In [ ]: def compute_P(C, nelec):  
        nbasis = C.shape[0]  
        nocc = nelec // 2  
        P = np.zeros((nbasis, nbasis))  
        for mu in range(nbasis):  
            for nu in range(nbasis):  
                total = 0.0  
                for i in range(nocc):
```

```

        total += C[mu, i] * C[nu, i]
    P[mu, nu] = 2.0 * total
    return P

```

The closed-shell density matrix is

$$P_{\mu\nu} = 2 \sum_{i=1}^{n_{\text{occ}}} C_{\mu i} C_{\nu i},$$

where $n_{\text{occ}} = N_e/2$ for RHF.

```

In [ ]: def build_J(P, eri):
    nbasis = P.shape[0]
    J = np.zeros((nbasis, nbasis))
    for mu in range(nbasis):
        for nu in range(nbasis):
            total = 0.0
            for lam in range(nbasis):
                for sig in range(nbasis):
                    total += P[lam, sig] * eri[mu, nu, lam, sig]
            J[mu, nu] = total
    return J

```

The Coulomb matrix is the density contracted with two-electron integrals:

$$J_{\mu\nu} = \sum_{\lambda\sigma} P_{\lambda\sigma} (\mu\nu|\lambda\sigma).$$

```

In [ ]: def build_K(P, eri):
    nbasis = P.shape[0]
    K = np.zeros((nbasis, nbasis))
    for mu in range(nbasis):
        for nu in range(nbasis):
            total = 0.0
            for lam in range(nbasis):
                for sig in range(nbasis):
                    total += P[lam, sig] * eri[mu, lam, nu, sig]
            K[mu, nu] = total
    return K

```

The exchange matrix uses a different contraction pattern:

$$K_{\mu\nu} = \sum_{\lambda\sigma} P_{\lambda\sigma} (\mu\lambda|\nu\sigma).$$

```

In [ ]: def build_Fock(Hcore, J, K):
    return Hcore + J - 0.5 * K

```

The Fock matrix in RHF is

$$F_{\mu\nu} = H_{\mu\nu} + J_{\mu\nu} - \frac{1}{2} K_{\mu\nu}.$$

```

In [ ]: def nuclear_repulsion_energy(atoms):
    E = 0.0
    n = len(atoms)
    for A in range(n):
        ZA = atoms[A].Z
        for B in range(A + 1, n):
            ZB = atoms[B].Z
            dx = atoms[A].x - atoms[B].x
            dy = atoms[A].y - atoms[B].y
            dz = atoms[A].z - atoms[B].z
            dx *= ANGSTROM_TO_BOHR
            dy *= ANGSTROM_TO_BOHR
            dz *= ANGSTROM_TO_BOHR

```

```

        R = np.sqrt(dx * dx + dy * dy + dz * dz)
        E += ZA * ZB / R
    return E

```

The nuclear repulsion energy is computed as

$$E_{\text{nuc}} = \sum_{A < B} \frac{Z_A Z_B}{R_{AB}},$$

with $R_{AB} = \|\mathbf{R}_A - \mathbf{R}_B\|$ in Bohr.

```

In [ ]: def electronic_energy(P, Hcore, F):
        return 0.5 * np.sum(P * (Hcore + F))

```

The electronic energy uses the standard RHF expression:

$$E_{\text{elec}} = \frac{1}{2} \sum_{\mu\nu} P_{\mu\nu} (H_{\mu\nu} + F_{\mu\nu}).$$

```

In [ ]: def scf_step(P, Hcore, eri, S_inv_sqrt, nelec):
        J = build_J(P, eri)
        K = build_K(P, eri)
        F = build_Fock(Hcore, J, K)
        Fp = S_inv_sqrt.T @ F @ S_inv_sqrt
        eps, Cp = np.linalg.eigh(Fp)
        C = S_inv_sqrt @ Cp
        P_new = compute_P(C, nelec)
        return F, C, eps, P_new

```

One SCF update performs orthonormalization and diagonalization:

$$F' = X^T F X \text{ with } X = S^{-1/2}$$

$$F' C' = C' \epsilon, \text{ then } C = X C'.$$

The new density is built as $P_{\mu\nu} = 2 \sum_i C_{\mu i} C_{\nu i}$.

```

In [ ]: def run_scf(
        mol,
        S,
        T,
        V,
        eri,
        max_iter=50,
        tol=1e-6,
        verbose=True,
    ):

```

This function receives the molecule, AO-basis integrals, and convergence controls, and returns a result dictionary for energies and orbitals.

```

In [ ]:     charge = mol.charge
        nelec = sum(a.Z for a in mol.atoms) - charge
        if nelec % 2 != 0:
            raise ValueError(f"RHF requires even nelec, got {nelec}")
        nocc = nelec // 2

        nbasis = S.shape[0]
        Hcore = build_Hcore(T, V)
        S_inv_sqrt = build_S_inv_sqrt(S)

```

The electron count is

$$N_e = \sum_A Z_A - q,$$

with $n_{\text{occ}} = N_e/2$ for RHF, and the core and orthonormalization matrices are prepared for the SCF loop.

```
In [ ]: E_nuc = nuclear_repulsion_energy(mol.atoms)
        F_core = Hcore
        Fp_core = S_inv_sqrt.T @ F_core @ S_inv_sqrt
        eps_core, Cp_core = np.linalg.eigh(Fp_core)
        C_core = S_inv_sqrt @ Cp_core
        P = compute_P(C_core, nelec)
```

The initial guess diagonalizes the core Hamiltonian:

$$F'_{\text{core}} = X^T H_{\text{core}} X,$$

$$F'_{\text{core}} C' = C' \varepsilon, \text{ then } C = XC'.$$

The starting density is $P_{\mu\nu} = 2 \sum_i C_{\mu i} C_{\nu i}$.

```
In [ ]: if verbose:
        print(f"SCF start: charge={charge}, nelec={nelec}, nocc={nocc}, nbasis={nbasis}")
        print(f"E_nuc = {E_nuc:.10f}")
        print("iter  max|ΔP|          E_elec          E_tot")
```

When verbose, the code prints the iteration header and the nuclear repulsion energy before the loop.

```
In [ ]: E_elec_print = 0.0
        E_tot_print = E_nuc

        converged = False

        for it in range(1, max_iter + 1):
            F, C, eps, P_new = scf_step(P, Hcore, eri, S_inv_sqrt, nelec)

            deltaP = np.max(np.abs(P_new - P))

            J_energy = build_J(P_new, eri)
            K_energy = build_K(P_new, eri)
            F_energy = build_Fock(Hcore, J_energy, K_energy)
            E_elec = electronic_energy(P_new, Hcore, F_energy)
            E_tot = E_elec + E_nuc

            if verbose:
                print(f"{it:4d}  {deltaP: .10f}  {E_elec_print: .10f}  {E_tot_print: .10f}")

            E_elec_print = E_elec
            E_tot_print = E_tot

            if deltaP < tol and it > 1:
                converged = True
                P = P_new
                break

        P = P_new
```

Each iteration updates the density and checks convergence using

$$\Delta P = \max_{\mu\nu} |P_{\mu\nu}^{(k)} - P_{\mu\nu}^{(k-1)}|.$$

Energies are updated with $E_{\text{tot}} = E_{\text{elec}} + E_{\text{nuc}}$, where $E_{\text{elec}} = \frac{1}{2} \sum_{\mu\nu} P_{\mu\nu} (H_{\mu\nu} + F_{\mu\nu})$.

```
In [ ]: F, C, eps, _ = scf_step(P, Hcore, eri, S_inv_sqrt, nelec)
        E_elec = electronic_energy(P, Hcore, F)
        E_tot = E_elec + E_nuc

        return {
            "P": P,
            "F": F,
            "C": C,
            "eps": eps,
            "E_elec": E_elec,
            "E_tot": E_tot,
            "E_nuc": E_nuc,
            "niter": it,
            "converged": converged,
        }
```

After convergence (or the last cycle), the final matrices and energies are recomputed and returned in a dictionary.

gradient.py

`gradient.py` computes the RHF nuclear gradients by combining integral derivatives with the SCF density matrices. The total gradient is assembled as

$$\nabla E = \nabla E_{\text{overlap}} + \nabla E_{\text{kinetic}} + \nabla E_{\text{nuc-attraction}} + \nabla E_{\text{two-e}} + \nabla E_{\text{nuc-rep}}.$$

```
In [ ]: import numpy as np

        from scf import ANGSTROM_TO_BOHR
```

NumPy is used for tensor operations. The Angstrom-to-Bohr factor ensures gradients are returned in Hartree/Bohr.

```
In [ ]: def build_Q(C, eps, nelec):
        if C is None or eps is None:
            raise ValueError("SCF coefficients/energies are required to build W.")

        nocc = nelec // 2
        C_mui = C[:, :nocc]
        eps_i = eps[:nocc]

        return 2.0 * C_mui @ np.diag(eps_i) @ C_mui.T
```

The energy-weighted density is

$$W_{\mu\nu} = 2 \sum_{i=1}^{n_{\text{occ}}} \varepsilon_i C_{\mu i} C_{\nu i},$$

with $n_{\text{occ}} = N_e/2$ for RHF.

```
In [ ]: def _contract_density_deriv(dX, density):
    natoms = dX.shape[0]
    nbasis = density.shape[0]
    grad = np.zeros((natoms, 3), dtype=float)

    for A in range(natoms):
        for mu in range(nbasis):
            for nu in range(nbasis):
                grad[A] += density[mu, nu] * dX[A, mu, nu]

    return grad
```

This contraction applies

$$\mathbf{g}_A = \sum_{\mu\nu} D_{\mu\nu} \frac{\partial X_{\mu\nu}}{\partial \mathbf{R}_A},$$

where D is a density-like matrix and X is an integral matrix (S , T , or V).

```
In [ ]: def nuclear_repulsion_gradient(atoms):
    natoms = len(atoms)
    grad = np.zeros((natoms, 3), dtype=float)

    coords = (
        np.array([[atom.x, atom.y, atom.z] for atom in atoms], dtype=float)
        * ANGSTROM_TO_BOHR
    )
    charges = np.array([atom.Z for atom in atoms], dtype=float)

    for A in range(natoms):
        RA = coords[A]
        ZA = charges[A]
        for B in range(natoms):
            if A == B:
                continue
            RB = coords[B]
            ZB = charges[B]

            R = RA - RB
            dist = np.linalg.norm(R)
            grad[A] += -ZA * ZB * R / (dist ** 3)

    return grad
```

The nuclear repulsion energy is

$$E_{\text{nuc}} = \sum_{A < B} \frac{Z_A Z_B}{R_{AB}},$$

and its gradient is

$$\nabla_A E_{\text{nuc}} = - \sum_{B \neq A} \frac{Z_A Z_B}{R_{AB}^3} (\mathbf{R}_A - \mathbf{R}_B),$$

with $\mathbf{R}$ in Bohr.

```
In [ ]: def compute_gradient(
    mol,
    scf_results,
    dS,
    dT,
    dV,
```



```

dERI,
):
    P = scf_results.get("P")
    C = scf_results.get("C")
    eps = scf_results.get("eps")

    nelec = sum(atom.Z for atom in mol.atoms) - mol.charge
    Q = build_Q(C, eps, nelec)

    grad_overlap = -_contract_density_deriv(dS, Q)
    grad_kinetic = _contract_density_deriv(dT, P)
    grad_nuclear_attr = _contract_density_deriv(dV, P)

    natoms = dERI.shape[0]
    nbasis = P.shape[0]
    grad_two_electron = np.zeros((natoms, 3), dtype=float)

    for A in range(natoms):
        for mu in range(nbasis):
            for nu in range(nbasis):
                for lam in range(nbasis):
                    for sig in range(nbasis):
                        pref = P[mu, nu] * P[lam, sig]
                        val1 = dERI[A, mu, nu, lam, sig]
                        val2 = dERI[A, mu, lam, nu, sig]
                        grad_two_electron[A, 0] += 0.5 * pref * val1[0]
                        grad_two_electron[A, 1] += 0.5 * pref * val1[1]
                        grad_two_electron[A, 2] += 0.5 * pref * val1[2]
                        grad_two_electron[A, 0] -= 0.25 * pref * val2[0]
                        grad_two_electron[A, 1] -= 0.25 * pref * val2[1]
                        grad_two_electron[A, 2] -= 0.25 * pref * val2[2]

    grad_nuclear_rep = nuclear_repulsion_gradient(mol.atoms)

    total = grad_overlap + grad_kinetic + grad_nuclear_attr + grad_two_electron + grad_nuclear_rep

    return {
        "overlap": grad_overlap,
        "kinetic": grad_kinetic,
        "nuclear_attraction": grad_nuclear_attr,
        "two_electron": grad_two_electron,
        "nuclear_repulsion": grad_nuclear_rep,
        "total": total,
    }

```

The overlap term is

$$\nabla E_{\text{overlap}} = - \sum_{\mu\nu} W_{\mu\nu} \frac{\partial S_{\mu\nu}}{\partial \mathbf{R}_A},$$

the kinetic and nuclear-attraction terms are

$$\nabla E_{\text{kinetic}} = \sum_{\mu\nu} P_{\mu\nu} \frac{\partial T_{\mu\nu}}{\partial \mathbf{R}_A},$$

$$\nabla E_{\text{nuc-attraction}} = \sum_{\mu\nu} P_{\mu\nu} \frac{\partial V_{\mu\nu}}{\partial \mathbf{R}_A}.$$

The two-electron contribution is evaluated as

$$\nabla E_{\text{two-e}} = \frac{1}{2} \sum_{\mu\nu\lambda\sigma} P_{\mu\nu} P_{\lambda\sigma} \frac{\partial}{\partial (\mu\nu|\lambda\sigma)} \frac{\partial}{\partial \mathbf{R}_A}$$

- $\frac{1}{4} \sum_{\mu \nu} \lambda_{\sigma} P_{\mu \nu} P_{\lambda \sigma} \frac{\partial}{\partial \lambda_{\sigma}} (\mu \lambda_{\nu} \sigma) \partial \mathbf{R}_A$.

main.py

In []: # main.py

```
import sys
from gradient import compute_gradient
from readinput import (
    read_basic_input,
    read_integrals,
    read_basis_block,
    read_derivatives,
)
from integrals import (
    compute_overlap,
    compute_kinetic,
    compute_nuclear_attraction,
    compute_two_electron,
    compute_overlap_derivatives,
    compute_kinetic_derivatives,
    compute_nuclear_attraction_derivatives,
    compute_two_electron_derivatives,
)
from scf import run_scf

def main():
    path = sys.argv[1]

    mol = read_basic_input(path)
    print("Number of atoms:", len(mol.atoms))
    print("Number of basis functions:", mol.nbasis)
    print("Total charge:", mol.charge)

    if "extended" in path.lower():
        print("Extended input detected.\n")

    S, T, V, eri = read_integrals(path, mol.nbasis)

    _, mu_to_atom = read_basis_block(path, mol.nbasis)
    dS, dT, dVder, dERI = read_derivatives(
        path,
        mol.nbasis,
        len(mol.atoms),
        mu_to_atom,
    )

    results = run_scf(
        mol,
        S,
        T,
        V,
        eri,
        max_iter=50,
        tol=1e-6,
        verbose=True,
    )

    print("\nSCF finished.")
    print("Converged:", results["converged"])
    print("Iterations:", results["niter"])
```

```

print("Final total energy:", results["E_tot"])
print("Final orbital energies:", results["eps"])

gradients = compute_gradient(
    mol,
    results,
    dS,
    dT,
    dVder,
    dERI,
)

def _print_block(title, data):
    print(title)
    for atom, vec in zip(mol.atoms, data):
        print(f"{atom.label:>2s} {vec[0]:.8f} {vec[1]:.8f} {vec[2]:.8f}")
    print()

print("\nEnergy gradient (Hartree/Bohr):")
_print_block("Total gradient:", gradients["total"])

print("Gradient contributions:")
_print_block(" - Overlap:", gradients["overlap"])
_print_block(" - Kinetic:", gradients["kinetic"])
_print_block(" - Electron-nuclear:", gradients["nuclear_attraction"])
_print_block(" - Two-electron:", gradients["two_electron"])
_print_block(" - Nuclear repulsion:", gradients["nuclear_repulsion"])

else:
    basis, _ = read_basis_block(path, mol.nbasis)
    S = compute_overlap(mol, basis)
    T = compute_kinetic(mol, basis)
    V = compute_nuclear_attraction(mol, basis)
    eri = compute_two_electron(mol, basis)

    results = run_scf(
        mol,
        S,
        T,
        V,
        eri,
        max_iter=50,
        tol=1e-6,
        verbose=True,
    )

    print("\nSCF finished.")
    print("Converged:", results["converged"])
    print("Iterations:", results["niter"])
    print("Final total energy:", results["E_tot"])
    print("Final orbital energies:", results["eps"])

    dS = compute_overlap_derivatives(mol, basis)
    dT = compute_kinetic_derivatives(mol, basis)
    dVder = compute_nuclear_attraction_derivatives(mol, basis)
    dERI = compute_two_electron_derivatives(mol, basis)

    gradients = compute_gradient(
        mol,
        results,
        dS,
        dT,
        dVder,
        dERI,
    )

```

```

def _print_block(title, data):
    print(title)
    for atom, vec in zip(mol.atoms, data):
        print(f"{atom.label:>2s} {vec[0]: .8f} {vec[1]: .8f} {vec[2]: .8f}")
    print()

print("\nEnergy gradient (Hartree/Bohr):")
_print_block("Total gradient:", gradients["total"])

print("Gradient contributions:")
_print_block(" - Overlap:", gradients["overlap"])
_print_block(" - Kinetic:", gradients["kinetic"])
_print_block(" - Electron-nuclear:", gradients["nuclear_attraction"])
_print_block(" - Two-electron:", gradients["two_electron"])
_print_block(" - Nuclear repulsion:", gradients["nuclear_repulsion"])

if __name__ == "__main__":
    main()

```

This script is the entry point. It reads the input file, prepares integrals, runs SCF, and computes nuclear gradients. The workflow branches depending on whether the filename contains `extended`.

General flow:

1. `read_basic_input` loads geometry, charge, and basis size and prints a short summary.
2. If `extended` is present in the filename, integrals and their derivatives are read from the file.
3. `run_scf` is called with `S`, `T`, `V`, and `eri`, and the SCF summary is printed.
4. `compute_gradient` builds the total gradient and prints each contribution.

Extended input path:

- `read_integrals` loads `S`, `T`, `V`, `eri` from the input.
- `read_basis_block` and `read_derivatives` load `dS`, `dT`, `dV`, `dERI`.

Non-extended input path:

- The basis is built with `read_basis_block`.
- One and two-electron integrals are computed with `integrals.py`.
- Derivatives are computed analytically before gradient evaluation.

Printed output:

- SCF convergence, iterations, total energy, and orbital energies.
- Total gradient in Hartree/Bohr.
- Individual contributions: overlap, kinetic, nuclear attraction, two-electron, and nuclear repulsion.